

ASSESSMENT TOOL 1-Constraint Based Problem Solving

ITA0401-Statistics with R Programming

S. Sharon Surya

192324246

Question 1. Student Marks Analysis (Vectors)

Pseudocode

Start

Store marks of 10 students in a vector.

Assign the first mark as highest and lowest.

Initialize sum as 0.

For each mark in the vector:

 If mark is greater than highest, update highest.

 If mark is less than lowest, update lowest.

 Add mark to sum.

Calculate average = sum / number of students.

Display highest mark.

Display lowest mark.

Display average mark.

Stop

CODE:

```
marks <- c(78,85,92,67,88,74,95,81,69,90)
```

```
highest <- marks[1]
```

```
lowest <- marks[1]
```

```
sum_marks <- 0
```

```
for(i in 1:length(marks))
```

```

{
  if(marks[i] > highest)
  {
    highest <- marks[i]
  }

  if(marks[i] < lowest)
  {
    lowest <- marks[i]
  }

  sum_marks <- sum_marks + marks[i]
}

average <- sum_marks/length(marks)

highest
lowest
average

```

OUTPUT:

```

> marks <- c(78, 85, 92, 67, 88, 74, 95, 81, 69, 90)
>
> highest <- marks[1]
> lowest <- marks[1]
> sum_marks <- 0
>
> for(i in 1:length(marks)){
+   if(marks[i] > highest){
+     highest <- marks[i]
+   }
+   if(marks[i] < lowest){
+     lowest <- marks[i]
+   }
+   sum_marks <- sum_marks + marks[i]
+ }
>
> average <- sum_marks/length(marks)
>
> highest
[1] 95
> lowest
[1] 67
> average
[1] 81.9
>

```

Constraints:

- Do not use built-in functions like sort().
- Use only vector indexing and arithmetic operators.
- Display the highest, lowest, and average marks.

Question 2. Employee Salary Matrix**Pseudocode**

Start

Create a 4 × 3 salary matrix.

Calculate department-wise total salary.

Find the department having maximum salary.

Display total salary.

Display department number.

Stop

CODE:

```
salary <- matrix(c(
30000,5000,4000,
32000,5500,4200,
28000,4500,3800,
35000,6000,4500
),nrow=4,byrow=TRUE)

total <- salary[,1] + salary[,2] + salary[,3]

total

which.max(total)
```

OUTPUT:

```
R 4.5.2 · ~/
> salary <- matrix(c(
+   30000,5000,4000,
+   32000,5500,4200,
+   28000,4500,3800,
+   35000,6000,4500
+ ),nrow=4,byrow=TRUE)
>
> total <- salary[,1] + salary[,2] + salary[,3]
>
> total
[1] 39000 41700 36300 45500
>
> which.max(total)
[1] 4
>
```

Constraints:

- Do not create separate vectors.
- Calculate total salary department-wise using matrix operations only.
- Display the department with the maximum total salary.

Question 3. Rainfall Data (Arrays)

Pseudocode:

Start

Create a three-dimensional rainfall array.

Initialize annual rainfall for each city.

For every city:

For every season:

For every year:

Add rainfall.

Find the city with highest rainfall.

Display annual rainfall.

Display city number.

Stop

CODE:

```
rainfall <- array(c(
120,150,130,140,
110,160,125,135,
```

```
100,140,120,150,
105,145,115,155,
```

```
130,170,140,160,
125,165,135,155
),dim=c(3,4,2))
```

```
annual <- c(0,0,0)
```

```
for(i in 1:3)
```

```
{
  total <- 0
```

```
  for(j in 1:4)
```

```
  {
    for(k in 1:2)
    {
      total <- total + rainfall[i,j,k]
    }
  }
```

```
  annual[i] <- total
```

```
}
```

```
annual
```

```
which.max(annual)
```

```
rainfall <- array(c(
```

```
120,150,130,140,
```

```
110,160,125,135,
```

```
100,140,120,150,
```

```
105,145,115,155,
```

```
130,170,140,160,
```

```
125,165,135,155
```

```
),dim=c(3,4,2))
```

```
annual <- c(0,0,0)
```

```
for(i in 1:3)
```

```
{
```

```
  total <- 0
```

```
  for(j in 1:4)
```

```
  {
```

```
    for(k in 1:2)
```

```
    {
```

```
      total <- total + rainfall[i,j,k]
```

```
    }
```

```
  }
```

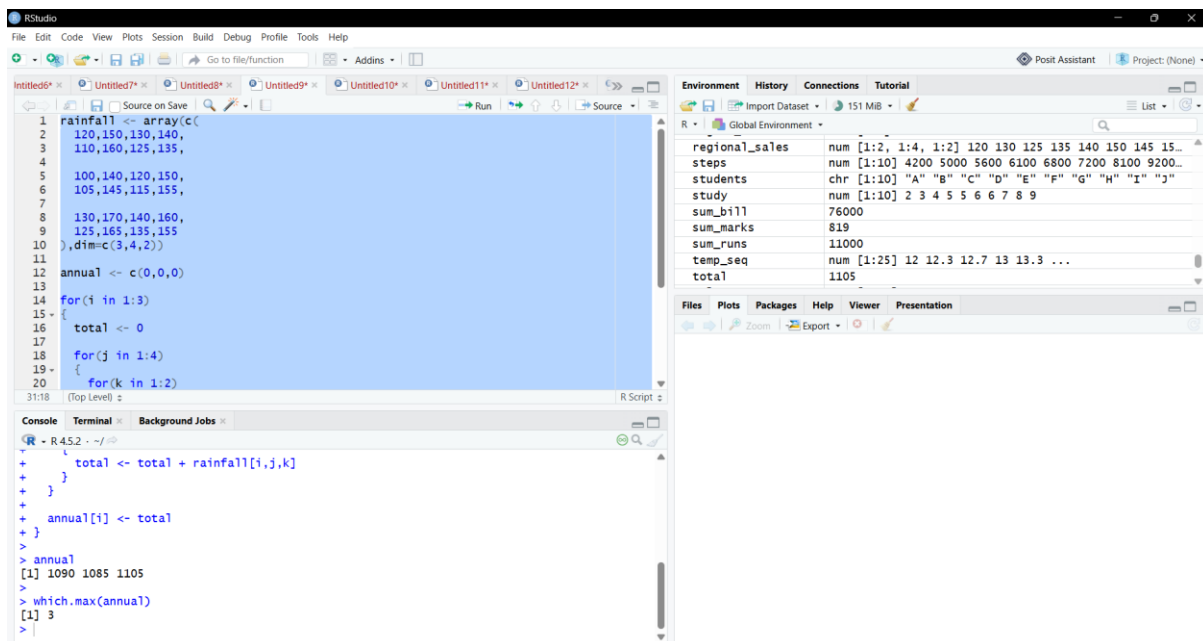
```
  annual[i] <- total
```

```
}
```

annual

which.max(annual)

OUTPUT:



```
1 rainfall <- array(c(
2   120,150,130,140,
3   110,160,125,135,
4   100,140,120,150,
5   105,145,115,155,
6   130,170,140,160,
7   125,165,135,155
8 ),dim=c(3,4,2))
9
10 annual <- c(0,0,0)
11
12 for(i in 1:3)
13 {
14   total <- 0
15   for(j in 1:4)
16   {
17     for(k in 1:2)
18     {
19       total <- total + rainfall[i,j,k]
20     }
21   }
22   annual[i] <- total
23 }
24
25 > annual
[1] 1090 1085 1105
26
27 > which.max(annual)
[1] 3
28
```

Constraints:

- Use only array indexing.
- Find the city with the highest annual rainfall.
- Do not convert the array into a matrix or data frame.

Question 4. Hospital Patient Records (Lists)

Pseudocode:

Start

Create a list containing patient details.

Identify patients whose age is above 60.

Display their ID, Name and Age.

Calculate total hospital bill.

Calculate average bill.

Display average bill.

Stop

CODE:

```
patients <- list(  
  ID=c(101,102,103,104,105),  
  Name=c("Arun","Bala","Cathy","David","Eva"),  
  Age=c(45,68,72,39,65),  
  BloodGroup=c("A+","B+","O+","AB+","A-"),  
  Bill=c(12000,18000,22000,9000,15000)  
)
```

```
index <- patients$Age > 60
```

```
patients$ID[index]
```

```
patients$Name[index]
```

```
patients$Age[index]
```

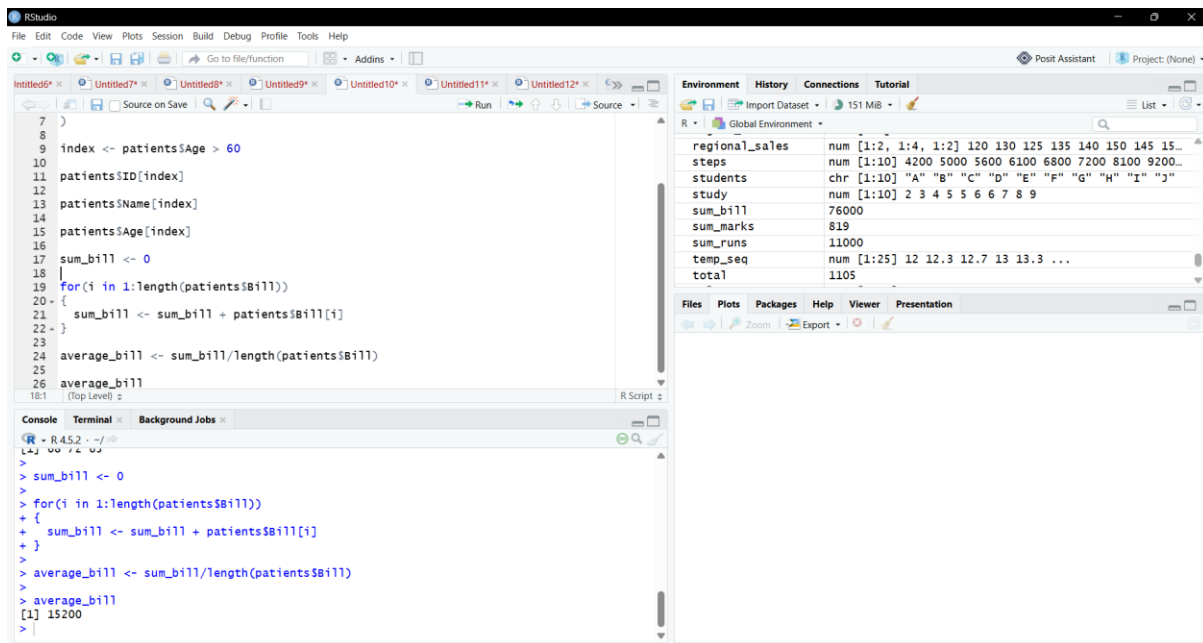
```
sum_bill <- 0
```

```
for(i in 1:length(patients$Bill))  
{  
  sum_bill <- sum_bill + patients$Bill[i]  
}
```

```
average_bill <- sum_bill/length(patients$Bill)
```


average_bill

OUTPUT:



Constraints:

- Store different data types in a single list.
- Retrieve only patients aged above 60.
- Calculate the average hospital bill without converting the list into another data structure.

Question 5. Product Inventory (Factors)

Pseudocode

Start

Create factor containing product categories.

Display categories in alphabetical order.

Initialize category counters.

Traverse each product.

Increase corresponding category count.

Display category count.

Stop

CODE:

```
products <- factor(c(
  "Electronics",
  "Grocery",
  "Stationery",
  "Electronics",
  "Grocery",
  "Grocery",
  "Stationery",
  "Electronics",
  "Stationery",
  "Grocery"
))

levels(products)

count <- c(0,0,0)

for(i in 1:length(products))
{
  if(products[i] == "Electronics")
  {
    count[1] <- count[1] + 1
  }
  else if(products[i] == "Grocery")
  {
    count[2] <- count[2] + 1
  }
  else
```

```

{
  count[3] <- count[3] + 1
}
}

```

Count

OUTPUT:

The screenshot shows the RStudio interface. The script editor contains the following code:

```

15 count <- c(0,0,0)
16
17 for(i in 1:length(products))
18 {
19   if(products[i] == "Electronics")
20   {
21     count[1] <- count[1] + 1
22   }
23   else if(products[i] == "Grocery")
24   {
25     count[2] <- count[2] + 1
26   }
27   else
28   {
29     count[3] <- count[3] + 1
30   }
31 }
32
33
34 count
35 (Top Level)

```

The Environment pane on the right shows the following objects:

Object	Class	Value
products	Factor w/ 3 levels "Electronics",...	1 2 3 1 2 2 3...
profit	num	[1:10] 20 25 32 40 42 55 65 72 82 95
projects	num	[1:10] 1 2 2 3 3 4 4 5 5 6
purchase	num	[1:10] 250 300 320 350 380 450 520 650 1800 52...
rainfall	num	[1:3, 1:4, 1:2] 120 150 130 140 110 160 125 13...
region_total	num	[1:2] 1020 1100
regional_sales	num	[1:2, 1:4, 1:2] 120 130 125 135 140 150 145 15...
steps	num	[1:10] 4200 5000 5600 6100 6800 7200 8100 9200...
students	chr	[1:10] "A" "B" "C" "D" "E" "F" "G" "H" "I" "J"

The Console shows the execution of the code and the final output:

```

> count
[1] 3 4 3

```

Constraints:

- Count the number of products in each category.
- Do not use table().
- Display categories in alphabetical order.

Question 6. Cricket Performance Analysis (Data Frames)

Pseudocode

Start

Create a data frame containing player details.

Select players with Strike Rate greater than 120.

Calculate total runs scored.

Compute average runs.

Display players with Strike Rate greater than 120.

Display average runs.

Stop

CODE:

```
players <- data.frame(  
  Name=c("Rohit","Virat","Gill","Hardik","Surya"),  
  Matches=c(50,60,40,35,45),  
  Runs=c(2500,3200,1800,1400,2100),  
  Wickets=c(2,5,0,45,1),  
  StrikeRate=c(125,118,130,128,145)  
)
```

```
players[players$StrikeRate>120,]
```

```
sum_runs <- 0
```

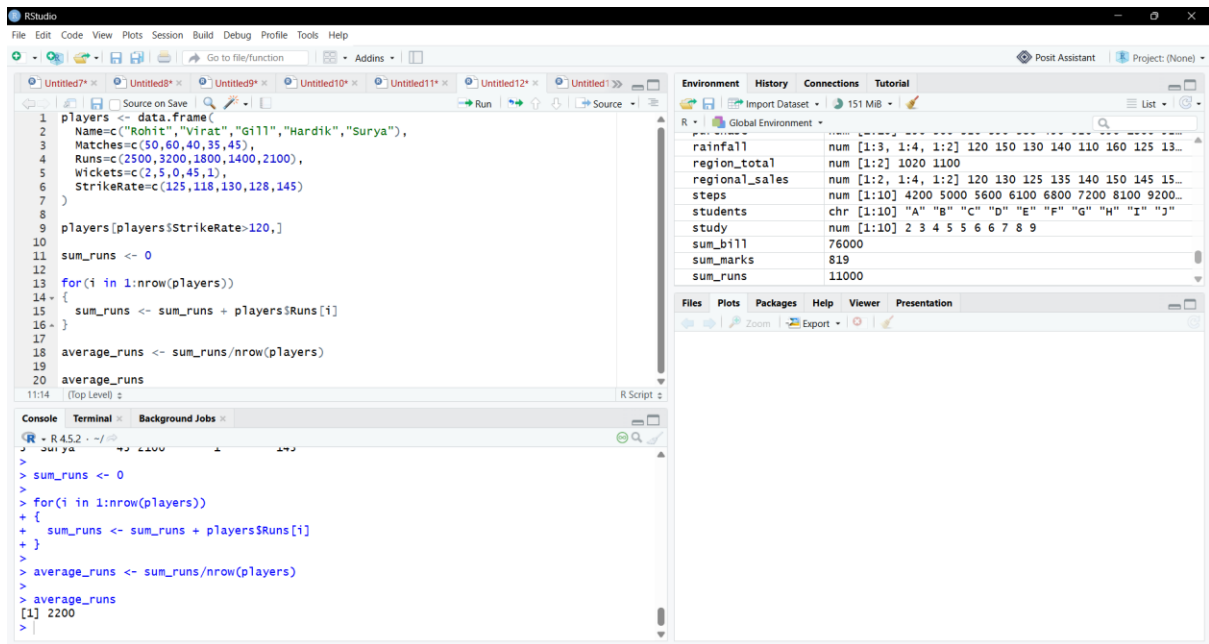
```
for(i in 1:nrow(players))
```

```
{  
  sum_runs <- sum_runs + players$Runs[i]  
}
```

```
average_runs <- sum_runs/nrow(players)
```

```
average_runs
```

OUTPUT:



Constraints:

- Identify players with Strike Rate greater than 120.
- Compute the average runs scored.
- Do not use SQL queries or external packages.

Question 7. Bank Transaction Processing (Class Types)

Pseudocode

Start

Create a data frame containing bank details.

Display the class of every column.

Convert account status into factor.

Display the class again.

Verify balance remains numeric.

Stop

CODE:

```

bank <- data.frame(
AccountNo=c(1001,1002,1003,1004),

```

```

Customer=c("Arun","Bala","Cathy","David"),
Balance=c(45000,38000,72000,29000),
Status=c("Active","Inactive","Active","Inactive")
)

```

```
sapply(bank,class)
```

```
bank$Status <- factor(bank$Status)
```

```
sapply(bank,class)
```

OUTPUT:

The screenshot shows the RStudio interface. The script editor on the left contains the following code:

```

1 bank <- data.frame(
2   AccountNo=c(1001,1002,1003,1004),
3   Customer=c("Arun","Bala","Cathy","David"),
4   Balance=c(45000,38000,72000,29000),
5   Status=c("Active","Inactive","Active","Inactive")
6 )
7
8 sapply(bank,class)
9
10 bank$Status <- factor(bank$Status)
11
12 sapply(bank,class)
13
14

```

The console on the bottom left shows the output of the code:

```

R - R4.5.2 ~ /...
> bank <- data.frame(
+   AccountNo=c(1001,1002,1003,1004),
+   Customer=c("Arun","Bala","Cathy","David"),
+   Balance=c(45000,38000,72000,29000),
+   Status=c("Active","Inactive","Active","Inactive")
+ )
>
> sapply(bank,class)
  AccountNo      Customer      Balance      Status
"numeric"  "character"  "numeric"  "character"
>
> bank$Status <- factor(bank$Status)
>
> sapply(bank,class)
  AccountNo      Customer      Balance      Status
"numeric"  "character"  "numeric"    "factor"
>

```

The Environment pane on the right shows the objects created in the global environment:

- bank: 4 obs. of 4 variables
- city: 5 obs. of 5 variables
- consumer: 5 obs. of 3 variables
- cor_matrix: num [1:3, 1:3] 1 -0.914 0.178 -0.914 1 ...
- correlation: num [1:3, 1:3] 1 0.915 1 0.915 1 ...
- data: 10 obs. of 5 variables
- direction: 8 obs. of 2 variables
- edges: 10 obs. of 2 variables
- electricity: num [1:5, 1:12] 220 180 260 200 240 230 190 270 ...

Constraints:

- Students with attendance below 85% are not eligible for examinations.
- Display eligible and ineligible students separately.
- Calculate the percentage of eligible students.

Question 8. University Attendance Management (Vectors and Logical Operations)

Pseudocode

Start

Store attendance percentages in a vector.

Separate eligible students having attendance 85% or above.

Separate ineligible students having attendance below 85%.

Calculate percentage of eligible students.

Display eligible students.

Display ineligible students.

Display eligibility percentage.

Stop

CODE:

```
attendance <- c(90,82,87,91,79,88,84,95,86,80,92,89,83,97,85)
```

```
eligible <- attendance[attendance>=85]
```

```
ineligible <- attendance[attendance<85]
```

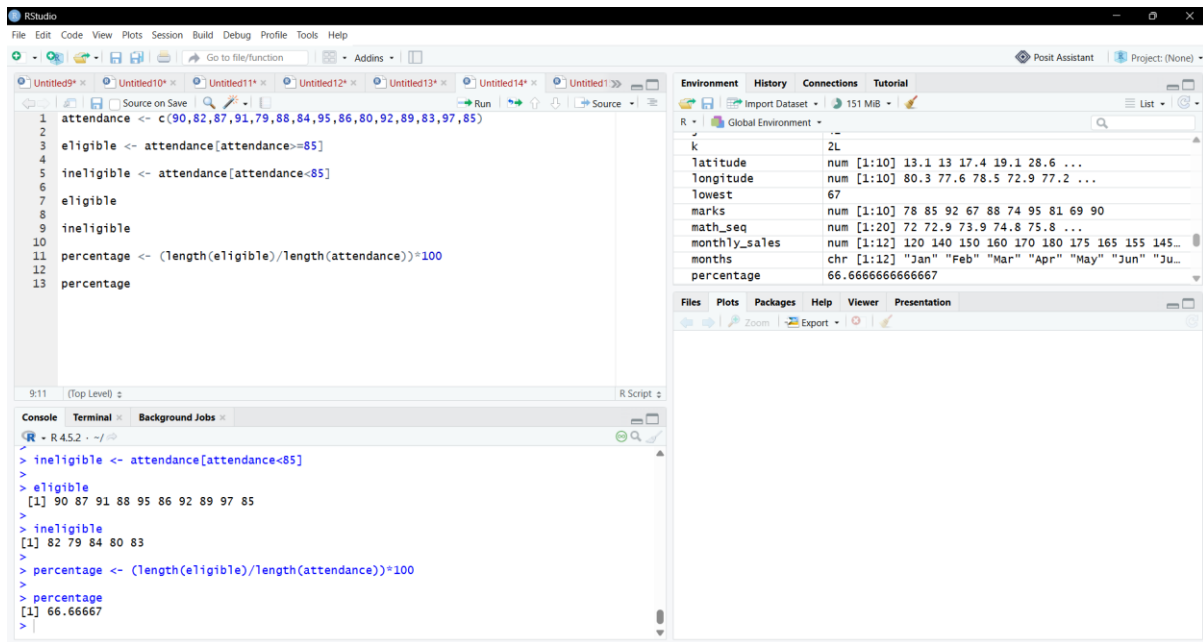
```
eligible
```

```
ineligible
```

```
percentage <- (length(eligible)/length(attendance))*100
```

```
percentage
```

OUTPUT:



Constraints:

- Students with attendance below 85% are not eligible for examinations.
- Display eligible and ineligible students separately.
- Calculate the percentage of eligible students.

Question 9. Electricity Consumption Analysis (Matrices)

Pseudocode

Start

Create a matrix for monthly electricity consumption.

Calculate annual consumption of each building.

Find the building with the highest annual consumption.

Display annual consumption.

Display building number.

Stop

Code

```
electricity <- matrix(c(
220,230,210,240,250,260,255,245,235,225,215,205,
180,190,200,210,220,230,225,215,205,195,185,175,
260,270,280,290,300,310,305,295,285,275,265,255,
```



```

200,210,220,230,240,250,245,235,225,215,205,195,
240,250,260,270,280,290,285,275,265,255,245,235
),nrow=5,byrow=TRUE)

```

```

annual <- electricity %*% rep(1,12)

```

```

annual

```

```

which.max(annual)

```

OUTPUT:

The screenshot shows the RStudio interface with the following content:

```

1 electricity <- matrix(c(
2   220,230,210,240,250,260,255,245,235,225,215,205,
3   180,190,200,210,220,230,225,215,205,195,185,175,
4   260,270,280,290,300,310,305,295,285,275,265,255,
5   200,210,220,230,240,250,245,235,225,215,205,195,
6   240,250,260,270,280,290,285,275,265,255,245,235
7 ),nrow=5,byrow=TRUE)
8
9 annual <- electricity %*% rep(1,12)
10
11 annual
12
13 which.max(annual)

```

The console output shows the result of the calculations:

```

> annual
[1,]
[1,] 2790
[2,] 2430
[3,] 3390
[4,] 2670
[5,] 3150
> which.max(annual)
[1] 3
>

```

The Environment pane on the right shows the following objects:

- electricity: num [1:5, 1:12] 220 180 260 200 240 230 190 270 ...
- engagement_data: 7 obs. of 4 variables
- env: 5 obs. of 4 variables
- fig: List of 8
- fig1: List of 8
- fig2: List of 8
- finance: 5 obs. of 3 variables
- finance_long: 15 obs. of 2 variables

Constraints:

- Calculate annual consumption for each building.
- Identify the building with the highest consumption.
- Use matrix operations only.

Question 10. Sales Performance Dashboard (Vectors, Matrices, Arrays and Class Types)

Pseudocode

Start

Create monthly sales vector.

Create quarterly sales matrix.

Create regional sales array.

Create product categories as factors.

Display the class of each object.

Calculate total annual sales.

Calculate total sales for each region.

Find the best-performing region.

Display annual sales.

Display region totals.

Display best-performing region.

Display quarterly sales.

Stop

Code

```
monthly_sales <- c(120,140,150,160,170,180,175,165,155,145,135,125)
```

```
quarterly_sales <- matrix(c(  
  410,450,495,405,  
  430,470,500,420  
),nrow=2,byrow=TRUE)
```

```
regional_sales <- array(c(  
  120,130,125,135,  
  140,150,145,155,  
  
  110,120,115,125,  
  130,140,135,145
```

```
),dim=c(2,4,2))
```

```
categories <- factor(c(  
  "Electronics",  
  "Grocery",  
  "Stationery",  
  "Electronics",  
  "Grocery"  
))
```

```
class(monthly_sales)
```

```
class(quarterly_sales)
```

```
class(regional_sales)
```

```
class(categories)
```

```
annual_sales <- 0
```

```
for(i in 1:length(monthly_sales))  
{  
  annual_sales <- annual_sales + monthly_sales[i]  
}
```

```
annual_sales
```

```
region_total <- c(0,0)
```

```
for(i in 1:2)  
{
```

```

total <- 0

for(j in 1:4)
{
  for(k in 1:2)
  {
    total <- total + regional_sales[i,j,k]
  }
}

region_total[i] <- total
}

```

region_total

which.max(region_total)

quarterly_sales

OUTPUT:

The screenshot shows the RStudio interface with the following components:

- Source Editor:** Contains the R script code for calculating regional totals and identifying the maximum.
- Console:** Displays the execution results of the code, including the final values of `region_total`, the index of the maximum value, and the `quarterly_sales` data frame.
- Environment:** Lists the objects in the global environment, including `regional_sales`, `steps`, `students`, `study`, `sum_bill`, `sum_marks`, `sum_runs`, `temp_seq`, and `total`.

Source Editor Code:

```

43 for(i in 1:2)
44 {
45   total <- 0
46   for(j in 1:4)
47   {
48     for(k in 1:2)
49     {
50       total <- total + regional_sales[i,j,k]
51     }
52   }
53   region_total[i] <- total
54 }
55
56 region_total
57
58 which.max(region_total)
59
60 quarterly_sales
61
62

```

Console Output:

```

> region_total
[1] 1020 1100
> which.max(region_total)
[1] 2
> quarterly_sales
  [,1] [,2] [,3] [,4]
[1,] 410 450 495 405
[2,] 430 470 500 420

```

Environment:

Object	Class	Dimensions	Values
regional_sales	num	[1:2, 1:4, 1:2]	120 130 125 135 140 150 145 15...
steps	num	[1:10]	4200 5000 5600 6100 6800 7200 8100 9200...
students	chr	[1:10]	"A" "B" "C" "D" "E" "F" "G" "H" "I" "J"
study	num	[1:10]	2 3 4 5 5 6 6 7 8 9
sum_bill	num		76000
sum_marks	num		819
sum_runs	num		11000
temp_seq	num	[1:25]	12 12.3 12.7 13 13.3 ...
total	num		1100

Constraints:

- Compute total annual sales.
- Identify the best-performing region.
- Compare quarterly performance.
- Verify the class type of every object before performing calculations.
- Use only base R functions.